

AUTOMATING WEB FORM AND SERVER-SIDE CODE GENERATION

Created by: *Muhammad Qasim Ali Rasheed (mr224su@student.lnu.se)*

Objective

- **Which domain and problem will you address?**

There are two domains:

- The **Abstract User Interface (UI)** domain, which describes the logical components of a form (e.g., text inputs, checkboxes) without specifying visual details.
- The **Web Development** domain, which includes the concrete technologies used to build a web application (HTML for the front end and Java for the backend).

The problem that is being addressed is the process of manually creating HTML forms and the corresponding Java classes to handle the submitted data which is repetitive and error prone. When a form gets changed or updated (e.g., a new field is added), developers need to update both the HTML file and the Java file. This project aims to solve this by generating both artifacts from a single, abstract model, which will ensure that they always remain synchronized and correct.

Models

- **What models and metamodels are needed?**

I foresee the use of two metamodels (Figure 1 depicts them):

- A metamodel that can represent the abstract structure of a web form (**FormMM**).
- A metamodel that can represent the structure of a Java class (**JavaClassMM**).

- **How many domains, MMs, and their relations.**

There are two domains, as described in the objective. The metamodels are related in the following way:

- The **FormMM** is in charge of representing everything in the abstract UI domain (forms, text fields, checkboxes, labels, etc.).

- The **JavaClassMM** is in charge of representing the structure of a Java class (classes, fields, data types).
- The two metamodels are independent. The connection between them is created by an M2M transformation, which will read a model that conforms to the FormMM and generate a new model that conforms to the JavaClassMM.

- **Tool integration - external tools that produce or consume models.**

This project does not require integration with an external mathematical or scheduling tool. The tools that consume the final output will be standard development tools i.e., A web browser and the Java Development Kit (JDK

The goal is to generate artifacts that are directly usable within a standard web development environment.

- **How will the model be created, updated?**

- The **Form model** will be created and updated manually by the developer. This model is the single source of truth for the entire system. Any change to the form's structure is made only in this file.
- The **JavaClass model** is created automatically from an M2M transformation. It should never be edited by hand, as it is an intermediate artifact.
- The final **HTML and Java files** are updated automatically every time their respective M2T transformations are executed.

Transformations

- **What transformation will you develop?**

I will develop at least three transformations.

1. **M2M Transformation (Form Model -> JavaClass Model):** This transformation takes the high-level, abstract definition of a form and translates it into a structured, logical representation of a Java data object. This creates an intermediate model that is purely focused on the data structure.
2. **M2T Transformation (Form Model -> HTML File):** This transformation is for creating the HTML code for user facing part of application. It takes the abstract Form Model and generates a.html file. It will iterate through the form elements defined in the model and produce the corresponding HTML tags (<form>, <input>, <label>, etc.

3. **M2T Transformation (JavaClass Model -> Java File):** This transformation creates backend logic. It takes the JavaClass Model (which was generated by the M2M step) and produces a syntactically correct .java source file. It will generate the class definition, private fields for each form element, and public getter/setter methods, creating a standard Plain Old Java Object that can be easily integrated into a larger Java application.

- **How will you combine transformations?**

- The **M2M (Form to JavaClass)** transformation and the **M2T (JavaClass to Java)** transformation must execute sequentially. The second transformation is dependent on the output model created by the first.
- The **M2T (Form to HTML)** transformation executes independently of the other two. It only requires the initial Form Model as input and can be run at any time, even in parallel with the Java-related transformations.

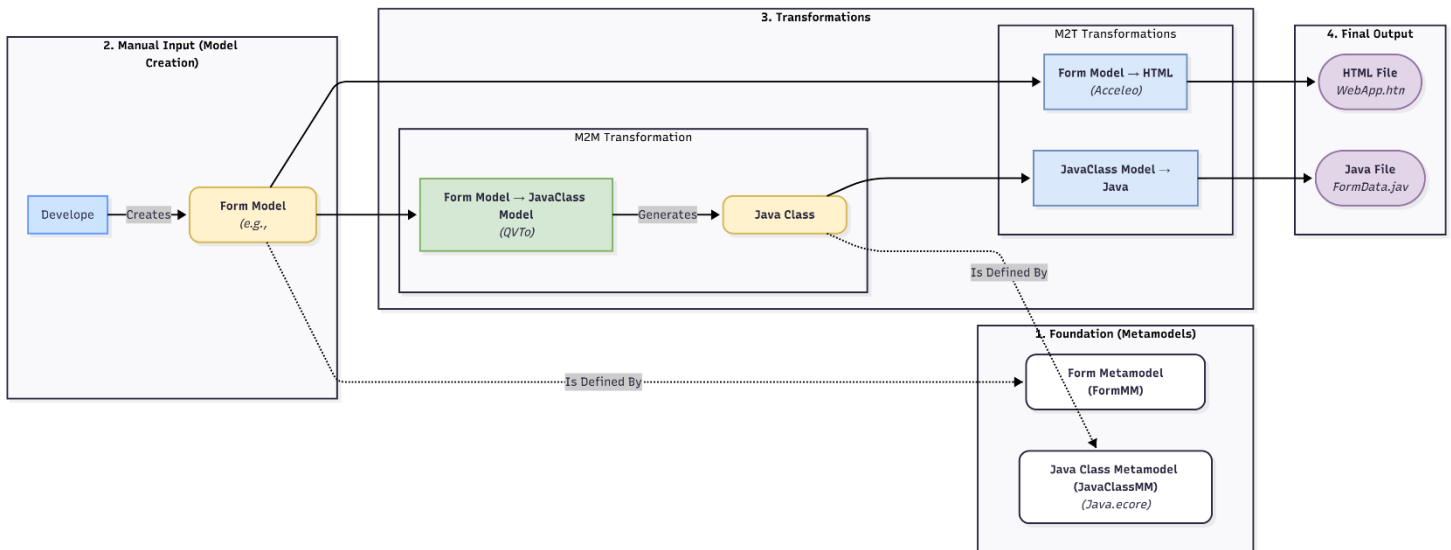


Figure 1